

LOCAL AUTONOMOUS INTELLIGENCE AND REASONING SYSTEM

LAIRS

Unified Technical Standard

VERSION	1.0
DATE	February 18, 2026
AUTHOR	Russell Aydelotte
ORGANIZATION	Aegis Secure Convergence Systems
CLASSIFICATION	Public — Not Confidential — Not Proprietary
LICENSE	Creative Commons BY-NC-SA 4.0
ARCH REPOSITORY	github.com/ASConvergenceSystems/lairs-architecture-pub
GAE REPOSITORY	github.com/ASConvergenceSystems/governed-adversarial-evolution

This document is the open, non-proprietary convergence of the LAIRS Runtime Governance Architecture and LAIRS Audit Fabric Standard into a single public-release technical reference. All content approved for unrestricted distribution under the license stated above.

Intelligence is local. Authority remains human. Evolution is deliberate.

- **PLATFORM STATUS — RESEARCH PROTOTYPE**
- LAIRS is an active research and architectural exploration platform. Not production ready.
- Intended for controlled experimentation only. Components marked In Development are under active work.
- Do not deploy in production environments without explicit authorization from the Human Master Administrator.

TABLE OF CONTENTS

PA RT	SECTIO N	TITLE	TOPIC
I	1–12	Runtime Governance Architecture	Definition · GAT · Governance Layers · GAE · Compliance
II	13–24	Audit Fabric Standard	Merkle Ledger · SentiNet · Storage Tiers · Threat Model
III	25	Component Catalog	Implemented & In-Development Components
IV	26–33	Deployment & Technical Reference	Node Topology · Event Structure · Verification · Limitations
—	—	Appendix: Terminology	Glossary of Key Terms
—	—	Appendix: References	Source Documents & Repositories

LICENSE

Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International

Copyright © 2026 Russell Aydelotte, Aegis Secure Convergence Systems. This work is licensed under CC BY-NC-SA 4.0. You are free to share and adapt the material for non-commercial purposes under the same license, with appropriate attribution. Full license: <https://creativecommons.org/licenses/by-nc-sa/4.0>

PART I

Runtime Governance Architecture

Defines the LAIRS standard for Runtime-Governed Systems — computing environments in which authority, behavior, and state changes are continuously validated through cryptographic proof, independent verification, and enforceable policy constraints during operation. This part covers the governance model, required layers, artifact specifications, and compliance criteria.

§ 1

Definition

A **Runtime-Governed System** is a computing environment in which authority, behavior, and state changes are continuously validated through cryptographic proof, independent verification, and enforceable policy constraints during operation — not only at deployment time. Unlike traditional systems that rely on static configuration trust, runtime-governed systems maintain persistent assurance of correctness, authorization, and integrity while executing.

CORE PRINCIPLE

Trust is not assumed. Trust is continuously proven.

§ 2

Problems Addressed

Modern distributed and autonomous systems suffer from structural weaknesses that runtime governance directly addresses by introducing verifiable authority loops:

- Configuration and policy drift over time
- Loss of authority provenance for system actions
- Inability to independently verify state legitimacy
- Weak coupling between decision and enforcement layers
- Over-reliance on perimeter or identity-only security models
- Lack of provable governance for automated or AI-driven changes

§ 3

Runtime Governance Model — Three-Layer Architecture

The LAIRS architecture defines three primary governance layers, each with distinct responsibilities and a specific trust question answered at runtime.

LAYER	NAME	RESPONSIBILITIES	TRUST QUESTION
01 Authority	Metatron's Gate	Issue signed GATs · Immutable decision ledger · Quorum and approval requirements · Anchor trust roots · Audit provenance	"Who authorized this action?"
02 Fabric	SentiNet	Independent artifact verification · Node binding enforcement · Signature validation · Last-Known-Good state · Cross-node trust propagation	"Can this authority be trusted here?"
03 Enforcement	Sentinel Agents	Runtime policy enforcement · Integrity monitoring · Behavior control · Evidence generation · Local protective response	"Is the system doing what was authorized?"

§ 4

Governance Artifact Token (GAT)

A runtime-governed system must rely on explicit Governance Artifacts to authorize all actions. The **GAT (Governance Artifact Token)** is the formal LAIRS artifact specification. Without a valid, verified GAT, enforcement actions **must fail closed** — no exceptions.

PROPERTY	PURPOSE
Cryptographically signed	Tamper-evident authenticity from the issuing authority
Time bounded	Tokens expire — stale authority cannot be replayed
Context scoped	Authority limited to specific action types and contexts
Node bound	Tokens bind to specific infrastructure nodes or identities
Independently verifiable	Third-party validation without querying the issuer
Ledger anchored	Linked to an immutable decision log for provenance
Authority metadata	Records issuer, governing policy, and approval quorum
Action intent hash	Binds the token to the exact action being authorized

§ 5

Runtime Verification Requirements

A system qualifies as runtime-governed only if all of the following conditions hold *continuously* during operation:

- Authority artifacts are validated during execution — not only at deployment
- Verification occurs independently from the issuing authority
- Enforcement modules refuse actions when validation fails
- Expired or revoked authority cannot execute under any condition
- State rollback or Last-Known-Good (LKG) protection exists and is tested
- Evidence of every decision is recorded immutably
- Nodes can prove their operational state to external observers on demand

§ 6

Governed Adversarial Evolution (GAE)

LAIRS extends runtime governance through **Governed Adversarial Evolution (GAE)**, enabling continuous system improvement without sacrificing governance integrity. GAE defines the policy envelope within which adversarial and defensive activities may occur, with human approval gates required for all system changes.

COMPONENT	DESCRIPTION
Red Team Pressure	Adversarial testing inside defined policy envelopes
Blue Team Defense	Adaptive defense responses within governed boundaries
Purple Team Synthesis	Coordinated insight combining offensive and defensive findings
AI Advisory	AI recommendations without autonomous authority to act
Human Approval Gates	Cryptographic human sign-off required for all changes
Replayable Scenarios	Scenarios recorded for regression testing and auditability

Repository: github.com/ASConvergenceSystems/governed-adversarial-evolution

§ 7

Observe-Only Safety Mode

Runtime-governed systems support staged deployment through **Observe-Only Mode**, enabling trust calibration before full enforcement is activated. The current LAIRS deployment operates in this mode.

- Governance artifacts are issued but non-enforcing
- Verification pipeline is fully active and generating evidence
- Evidence is generated for all decisions — full audit trail active
- No destructive or state-modifying actions are permitted
- Provides a safe validation environment before live enforcement is enabled

§ 8

Properties of a Runtime-Governed System

A compliant system demonstrates all ten of the following properties:

PROPERTY	COMPLEMENTARY PROPERTY
Verifiable authority provenance	Fail-closed behavior under uncertainty
Continuous policy validation	Bounded autonomy
Cryptographic trust anchors	Human override capability
Independent enforcement capability	Recoverable safe state
Deterministic auditability	Cross-node trust coherence

§ 9

Comparison With Traditional Security Models

DIMENSION	TRADITIONAL SECURITY	LAIRS RUNTIME GOVERNANCE
Trust model	Static configuration trust	Continuous cryptographic verification
Authorization	Identity-based	Artifact-based authority (GAT)
Control topology	Centralized control plane	Distributed verification fabric
Audit timing	Logs after the fact	Proof generated during execution
Trust boundaries	Implicit and assumed	Explicit trust contracts (GAT-bound)
Failure behavior	Undefined / fail-open risk	Fail-closed by design
Adaptation	Reactive response	Governed adversarial evolution (GAE)

§ 10

Compliance Summary

A system meets the LAIRS Runtime Governance Standard if it satisfies all of the following criteria:

- Uses cryptographically signed governance artifacts (GATs) for all authorized actions
- Performs independent runtime verification across all enforcement nodes
- Enforces policy through verifiable agents with fail-closed behavior on artifact failure
- Maintains immutable decision provenance via cryptographically anchored ledger
- Supports safe recovery and Last-Known-Good (LKG) state rollback
- Enables governed evolution without authority loss through the GAE framework
- Implements multi-authority quorum governance for high-assurance decisions

§ 11

Benefits

- Reduced configuration drift risk through continuous artifact-based validation
- Provable compliance with immutable, independently verifiable decision records
- Safer automation and AI integration via bounded autonomy and human approval gates
- Strong forensic capability with evidence generated at every enforcement decision
- Independent trust validation without reliance on centralized authority at runtime
- Resilient distributed operation with cross-node trust coherence via SentiNet
- Controlled autonomy that evolves without sacrificing governance integrity

§ 12

Roadmap — Future Directions

- Hardware root attestation (TPM-backed key sealing + measured boot for silicon-level trust anchoring)
- Cross-organization trust federation enabling governed interoperability between separate LAIRS deployments
- Autonomous infrastructure governance for large-scale self-managing systems with human oversight retained
- Regulated AI control environments with LAIRS-compliant oversight layers and provable decision provenance
- Multi-peer consensus and Byzantine fault tolerance for SentiNet beyond two-node topology
- Formal verification proofs for governance protocol correctness and GAT lifecycle integrity

PART II

Audit Fabric Standard

Beyond Blockchain: Deterministic, Segmented, Cryptographically Anchored Logging for Governed Systems with Distributed Verification. This part introduces the LAIRS Audit Fabric Standard — a deterministic, cryptographically anchored audit architecture combining governance-aware events, segmented Merkle sealing, and bounded lifecycle storage, extended through SentiNet for distributed verification.

§ 13

Problem Statement

Current audit mechanisms fail to simultaneously satisfy the requirements of modern governed infrastructure. The following critical properties are absent from existing solutions:

- Tamper-evident continuity of operational history under adversarial conditions
- Governance-aware context embedded natively in audit records
- Bounded storage lifecycle with preserved cryptographic integrity
- Offline and independent verification capability without network access
- Distributed trust propagation across heterogeneous nodes
- Fail-closed integrity guarantees under partial failure conditions

Blockchain systems address decentralized trust but introduce consensus complexity, unbounded storage growth, and semantic mismatch unsuitable for infrastructure telemetry and runtime governance. The LAIRS Audit Fabric resolves these conflicts.

§ 14

Architectural Overview

The LAIRS Audit Fabric is composed of four integrated planes operating in a strict hierarchy of trust and enforcement:

PLANE	COMPONENT	ROLE
Enforcement	Sentinel Agent	Runtime policy enforcement and event generation
Governance	Metatron's Gate	Authority validation, GAT issuance, quorum decisions
Audit Fabric	Ledger + Merkle	Cryptographic event anchoring and segment sealing
Verification	SentiNet	Distributed trust propagation and node integrity

§ 15

System Pipeline — Deterministic Event Flow

Events flow through a deterministic, ordered pipeline from runtime origin to verified storage. Each stage produces verifiable output consumed by the next:

01	Runtime Event (Sentinel Agent generates enforcement event)	↓
02	Sentinel Enforcement (Policy evaluation — pass / fail / record)	↓
03	Metatron's Gate Governance Validation (GAT check, quorum verification)	■
01	Ledger Event Construction (Embed governance context, actor identity, intent)	↓
02	Hash Chain Continuity (Link to previous event hash — tamper-evident chain)	↓
03	Segmented Merkle Sealing (Compute Merkle tree over segment, generate root)	↓
04	Signed Artifact Storage (Digital signature applied, artifact committed)	↓
05	SentiNet Distribution (Push signed artifact to verification nodes)	↓
06	Node Verification & Enforcement (Independent signature and root validation)	↓
07	Tiered Storage Lifecycle (Hot → Warm → Cold with integrity preserved)	■

§ 16

Segmented Merkle Ledger Model

The LAIRS Audit Fabric uses segmented Merkle ledgers rather than monolithic append-only logs. Events are grouped into discrete segments that undergo a deterministic sealing process, producing portable, independently verifiable trust artifacts. This model enables partial verification of historical segments without requiring access to the full audit history:

- Merkle tree computation over all events within a segment
- Root generation producing a canonical segment fingerprint (SHA-256)
- Payload hashing binding compressed event content to the segment seal
- Digital signing by the governance authority (Metatron's Gate)
- Ledger commitment creating an immutable provenance anchor with prev_hash chain

§ 17

Governance-Coupled Event Semantics

Audit events in the LAIRS Fabric are not passive log entries. Each event carries embedded governance context, transforming logs into **decision provenance records**. This is a fundamental design distinction from conventional SIEM or syslog architectures.

FIELD	PURPOSE
Actor Identity	Cryptographically verified identity of the acting principal
Role Classification	Authority classification of the actor at time of action
Intent Metadata	Declared purpose of the action for governance traceability
Policy Constraints	Active policy envelope governing the specific action
Approval State	Quorum or unilateral approval status with signatory list
System Context	Runtime state snapshot captured at time of event creation

§ 18

Distributed Verification Fabric — SentiNet

SentiNet functions as the distributed trust propagation layer. Rather than relying on consensus-based ledgers, SentiNet propagates cryptographically signed state across nodes using a push-verify model with local enforcement capability. Each node independently validates trust state without querying the origin system:

- Signed policy bundles distributed to all enforcement nodes
- Integrity verification against known-good Merkle roots at each node
- Last Known Good (LKG) fallback automatically invoked on verification failure
- Fail-closed enforcement when trust state cannot be confirmed by any path
- Cross-node trust propagation without central coordination or consensus
- Independent audit segment verification at each node upon receipt

§ 19

Bounded Storage Lifecycle Model

The LAIRS Audit Fabric introduces bounded lifecycle storage across three tiers. Cryptographic integrity guarantees are preserved regardless of storage medium or access speed, enabling long-term forensic capability without unbounded growth:

TIER	CHARACTERISTICS	VERIFICATION METHOD
HOT	Active segments, immediate access, full event resolution	Real-time Merkle computation and hash chain
WARM	Sealed segments, compressed storage, recent history	Segment root verification against ledger
COLD	Archived artifacts, offline media, long-term retention	Offline signature validation and root check

§ 20

Verification and Replay

Independent validation is a first-class capability. Any authorized party can verify the integrity of historical records **without access to the originating system**. Verification is deterministic and comprises four steps:

- Segment signature verification against the issuing authority's public key
- Merkle root recomputation from raw event payloads — compare to ledger root
- Hash chain continuity check across all segment boundaries in sequence
- Governance metadata validation confirming policy envelope and approval state

Replay capability allows historical scenarios to be re-executed against current policy, supporting regression testing, incident investigation, and compliance auditing. Replay does not require access to the original runtime.

§ 21

Security Analysis — Threat Model

The LAIRS Audit Fabric provides cryptographic mitigations against a comprehensive infrastructure threat model:

THREAT	MITIGATION
Log tampering	Hash chain continuity + Merkle root binding detect any modification
Insider manipulation	Quorum approval requirements limit unilateral action; signed artifacts
Storage corruption	Segment-level Merkle sealing isolates and localizes corruption scope
Ledger truncation	Chain continuity gaps detected; SentiNet propagates last-good state
Replay attacks	Time-bounded GATs + segment sequence numbers prevent replay
Node compromise	Fail-closed enforcement + LKG fallback contain blast radius
Policy corruption	Signed policy bundles with independent verification detect tampering
Single-authority capture	Multi-party quorum required for all sensitive governance decisions

§ 22

Comparison With Blockchain

DIMENSION	BLOCKCHAIN	LAIRS AUDIT FABRIC
Trust model	Consensus-based	Cryptographic + governance-native
Storage growth	Unbounded by design	Bounded lifecycle tiers
Verification access	Requires live network	Fully offline capable
Governance coupling	External / implicit	Embedded in every event record
Latency profile	Consensus-dependent	Deterministic, local execution
Failure recovery	Fork resolution complexity	Last Known Good + fail-closed
Event semantics	Transaction-oriented	Decision provenance records
Infrastructure fit	Cryptocurrency / DeFi	Infrastructure telemetry + governance

§ 23

Implementation Evidence

THIS IS NOT THEORETICAL

The LAIRS Audit Fabric Standard has been implemented and the following capabilities are verified in active two-node deployment as of February 2026.

- Active governance events with embedded policy and approval metadata generated in production
- Merkle-sealed segments with independently verifiable root artifacts — validated
- Signed audit artifacts distributed across SentiNet nodes — operational
- Distributed policy verification without central coordination — confirmed
- Runtime enforcement integration with Sentinel Agent plane — active in observe mode

§ 24

Application Domains

The LAIRS Audit Fabric Standard is applicable across high-assurance environments where provable operational continuity and governed accountability are required:

- Critical infrastructure protection and operational continuity
- Industrial control systems (ICS/SCADA) requiring deterministic audit trails
- Defense networks with distributed, offline-capable verification requirements
- Autonomous AI governance with provable decision provenance and human override
- Sovereign cloud environments requiring fully offline verification capability
- Supply chain verification with cryptographic continuity and provenance guarantees

PART III

Component Catalog

Current public architecture as of February 18, 2026. Components classified as **Implemented** (operational in the current two-node deployment) or **In Development** (under active design or integration). The Human Master Administrator retains ultimate authority over all components at all times. No component may act autonomously or irreversibly without explicit human cryptographic approval.

IMPLEMENTED COMPONENTS

LAIRS Core

✓ IMPLEMENTED — OPERATIONAL

Role: Cognitive runtime and orchestration layer

- › Runtime coordination and event routing between all modules
- › Identity, memory, and state management
- › Policy hooks, audit logging, and explainability
- › Local-first execution — no external dependency for core operation
- › No autonomous irreversible actions without human approval

SentiNet Fabric

✓ IMPLEMENTED — OPERATIONAL

Role: Zero-trust secure networking and governance fabric

- › Zone-based segmentation with default-deny communication model
- › Internal DNS and PKI anchoring for verified node identity
- › Controlled bridge modes between trust zones — explicit policy required
- › Every module treated as untrusted until verified by SentiNet
- › Policy enforcement occurs at the fabric layer, not the application layer
- › Last Known Good (LKG) state validation and automatic fallback

Metatron's Gate

✓ IMPLEMENTED — OPERATIONAL

Role: Quorum governance and system authority control

- › Explicit intent declaration required for all sensitive actions
- › Multi-party quorum enforcement: Machine / AI / Human triad
- › Cryptographic approval and denial tracking with full audit trail
- › Observe-only and full enforcement operational modes
- › Prevents unilateral system modification by design — no single actor override
- › Append-only audit trail with causal chaining between decisions

Sentinel Agent

✓ IMPLEMENTED — OPERATIONAL

Role: Host-level integrity, telemetry, and local attestation

- › Host-level monitoring and integrity checks for runtime environment
- › Secure encrypted telemetry channel to LAIRS Core
- › Foundation module for trust establishment in the SentiNet fabric
- › Currently operating in observe mode — enforcement pending authorization

AiCraft

✓ IMPLEMENTED — OPERATIONAL

Role: Local AI training and lifecycle governance

- › Dataset ingestion and validation workflows with transparency logging
- › Model training and evaluation with versioned artifact output
- › Transparency into all training behavior — no hidden learning paths
- › Explicit datasets only — no passive or unsanctioned data incorporation
- › Human-directed experimentation with audit trail per training run

IN DEVELOPMENT

Sentinel Gate

● IN DEVELOPMENT

Role: Policy enforcement and controlled access (Gated Community)

- › Entry control point into SentiNet fabric — all ingress passes through Gate
- › Policy validation and trust scoring at ingress
- › Rejection handling for non-compliant or unverified traffic
- › Fail-closed by design — unknown traffic is denied by default

Sentinel AV

● IN DEVELOPMENT

Role: Malware detection and quarantine

- › Local scanning without external cloud dependency — fully sovereign
- › Quarantine workflows integrated with LKG state protection
- › Deep integration with Sentinel Agent and Sentinel Gate

Sentinel Forge

● IN DEVELOPMENT

Role: Human-approved governed system evolution

- › Detects issues, performance degradation, and candidate improvement opportunities
- › Generates candidate patches for human review — advisory only, never self-applies
- › Stages changes with full rollback capability before any application
- › Requires cryptographic human approval before any change is applied
- › Changes applied only at controlled reboot boundaries — no hot patching
- › Cannot self-modify autonomously under any condition

Leave No Trace (LNT)

● IN DEVELOPMENT

Role: Data minimization and forensic hygiene

- › Log scrubbing and configurable data retention control policies
- › Ephemeral runtime handling for sensitive operational data
- › Privacy-preserving operations that minimize persistent data footprint
- › Minimizes data footprint without compromising cryptographic audit integrity

PART IV

Deployment & Technical Reference

Technical specifications for the current two-node reference deployment, including node topology, ledger event structure, segment lifecycle, verification procedures, governance identity controls, SentiNet channel behavior, storage layout, security assumptions, and acknowledged limitations.

§ 25

Two-Node Deployment Topology

The current implementation operates across two physical systems connected through the SentiNet verification channel. Each node fulfills a distinct role in the governance and verification architecture, providing operational separation between artifact production and artifact verification:

GL360 NODE — PRIMARY	LENOVO NODE — SECONDARY
› Metatron's Gate governance engine › Segmented Merkle audit generation › Sentinel enforcement agent (observe mode) › Artifact sealing and archival storage › LAIRS Core runtime services › Produces authoritative audit artifacts. › Origin node for all governance decisions.	› SentiNet verification endpoint › Signed artifact validation against received root › Known-good state reference database › Independent verification environment › Provides verification independence › from the artifact origin environment.

§ 26

Ledger Event Structure

A representative SEGMENT_SEALED ledger event demonstrating the hash chain and Merkle commitment fields:

```
{
  "ts": 1739634561.123,
  "event_type": "SEGMENT_SEALED",
  "hash": "sha256:a3f9b12d...", // links this entry into hash chain
  "prev_hash": "sha256:7c4e0a91...", // continuity to previous entry
  "payload_sha256": "sha256:d1f8c322...", // protects compressed artifact
  "merkle_root": "sha256:9b2e7f55...", // commits segment event contents
  "segment_basename": "segment_20260218_145601.jsonl.gz",
  "sig_basename": "segment_20260218_145601.jsonl.gz.sig"
}
```


FIELD	PURPOSE & VERIFICATION ROLE
hash	Links the ledger entry into the hash chain; tamper detection
prev_hash	Provides continuity verification — gaps indicate truncation or insertion
merkle_root	Commits the full contents of the segment; root recomputation validates
payload_sha256	Protects the compressed artifact file against storage corruption
sig_basename	Points to detached signature enabling independent authenticity validation

§ 27

Segment Artifact Lifecycle

Each audit segment follows a deterministic, ordered lifecycle from event generation to archived storage. Each stage is atomic and produces verifiable output that feeds the next stage:

01	Events generated by governance engine (Metatron's Gate)	↓
02	Events written sequentially to segment JSONL file	↓
03	Merkle tree computed over all events in segment	↓
04	Merkle root hash generated — canonical segment fingerprint	↓
05	Segment payload compressed (gzip) and hashed (SHA-256)	↓
06	Digital signature applied by governance signing key	↓
07	Ledger entry recorded: hash, prev_hash, merkle_root, payload_sha256	↓
08	Artifact stored in audit archive: segment.jsonl.gz + segment.jsonl.gz.sig	■

ARTIFACT FILE FORMAT

Each sealed segment produces exactly two files: segment_YYYYMMDD_timestamp.jsonl.gz (compressed events) and segment_YYYYMMDD_timestamp.jsonl.gz.sig (detached signature). Both files are required for independent verification. Segments remain independently verifiable regardless of storage location.

§ 28

Independent Verification Procedure

The following procedure enables full independent verification of any historical segment. **No access to the original runtime system is required.** Only the segment files, signature files, and the authority's public key are needed:

- Decompress segment artifact: `gunzip segment_*.jsonl.gz → segment_*.jsonl`
- Recompute Merkle tree from all event records in the decompressed file
- Compare computed Merkle root to `merkle_root` value in `merkle_ledger.jsonl`
- Validate detached digital signature on `.sig` file against authority public key
- Confirm hash chain continuity: each entry's `prev_hash` == predecessor's hash
- Validate governance metadata fields (policy, approval state, actor identity) per event

§ 29

Governance Identity Controls

The governance identity layer implements authenticated attribution for all governance decisions. Current capabilities:

- Password hashing using `bcrypt` with per-credential salts
- Time-based one-time password (TOTP) MFA for all governance actors
- Enrollment validation workflows with identity event logging
- Policy-controlled authentication — access governed by active policy bundle
- Identity data bound to governance decisions in all audit records

LEGAL NOTE

The system provides authenticated attribution but does not claim legal non-repudiation guarantees. Consult appropriate legal counsel regarding regulatory requirements for your deployment context.

§ 30

SentiNet Verification Channel — Current Behavior

CURRENTLY OPERATIONAL	NOT YET IMPLEMENTED
› Signed artifact transfer between nodes › Signature verification at receiving node › Known-good state referencing › Validation independent of origin node	› Multi-peer consensus (>2 nodes) › Automatic mesh propagation › Byzantine fault tolerance › Automated cross-node distribution

SentiNet purpose: deterministic verification within a governed two-node trust boundary. Multi-node expansion is on the roadmap.

§ 31

Storage Layout

```
/mnt/sentinet_logs/
■■■ lairs_audit_archive/
■ ■■■ segment_20260218_000001.jsonl.gz # hot — active segments
■ ■■■ segment_20260218_000001.jsonl.gz.sig # detached signature
■ ■■■ segment_20260218_000002.jsonl.gz
■ ■■■ segment_20260218_000002.jsonl.gz.sig
■ ■■■ merkle_ledger.jsonl # hash chain ledger
■■■ [warm/cold archived via tiered lifecycle policy]
```

Integrity guarantees are preserved through cryptographic artifacts — not through storage medium trust. Cold artifacts on offline media retain full independent verifiability.

§ 32

Security Assumptions

The current implementation operates under the following security assumptions, consistent with enterprise infrastructure security models:

- Cryptographic primitives (SHA-256, RSA/ECDSA) are not compromised
- Governance signing keys are protected and not accessible to unauthorized parties
- Both nodes operate within a governed administrative boundary with physical controls
- Physical compromise of both nodes simultaneously is unlikely under the threat model

§ 33

Acknowledged Limitations of Current Implementation

The following limitations are acknowledged to prevent overstating system maturity. Each represents a known constraint of the current two-node research deployment:

■ CURRENT LIMITATIONS — RESEARCH DEPLOYMENT

- Two-node deployment only — not validated at scale, no multi-peer mesh
- Sentinel enforcement agent operating in observe mode — not active enforcement
- No hardware root-of-trust integration (TPM-backed key sealing planned)
- Limited automated distribution of signed artifacts between nodes
- No formal verification proofs for governance protocol or GAT lifecycle correctness

PLANNED: Hardware Root of Trust Integration

Future revisions will integrate TPM-backed key sealing and measured boot validation to bind governance signing authority to hardware state. This enables silicon-anchored artifact issuance and boot integrity attestation without altering the deterministic verification model described herein.

APPENDIX A

Terminology

TERM	DEFINITION
Audit Fabric	Cryptographically sealed, governance-aware logging system with deterministic verification. Produces decision provenance records, not passive log entries.
GAT	Governance Artifact Token. Cryptographically signed, time-bounded, context-scoped, node-bound artifact that authorizes a specific action. Required for all sensitive operations in a runtime-governed system. Absence or failure of GAT validation triggers fail-closed behavior.
GAE	Governed Adversarial Evolution. Framework enabling continuous system improvement through governed red/blue/purple team activities. All changes require cryptographic human approval gates.
Governance Engine	Component responsible for policy evaluation, GAT issuance, quorum enforcement, and decision recording. In LAIRS: Metatron's Gate.
Hash Chain	Sequence of cryptographic hashes where each entry includes the hash of its predecessor, providing tamper-evident continuity. Gaps indicate truncation or insertion.
LKG / Known-Good State	Last Known Good state. Previously validated system configuration used as fallback when current state verification fails. Enables recovery without loss of auditability.
Merkle Tree / Root	Binary hash tree structure over a set of events. The root hash cryptographically commits the entire set; any modification changes the root. Enables partial verification of segments without full history access.
Observe-Only Mode	Staged deployment mode in which governance artifacts are issued and the verification pipeline is fully active, but no destructive or state-modifying actions are permitted. Current operational mode of LAIRS deployment.
Runtime-Governed System	Computing environment where authority, behavior, and state changes are continuously validated through cryptographic proof, independent verification, and enforceable policy constraints during operation — not just at deployment.
Segment	Bounded, sealed group of audit events. Each segment has a Merkle tree computed over its events, a digital signature applied, and a ledger entry recording its root and hash chain position.
SentiNet	Distributed trust propagation and verification fabric. Provides zero-trust networking with zone-based segmentation, Last Known Good state validation, and signed artifact distribution across nodes.
Verification Channel	Mechanism used to distribute and validate signed artifacts across SentiNet nodes. Currently supports push-verify model with local enforcement; multi-peer mesh planned.

APPENDIX B

References & Source Documents

REF	TITLE	SOURCE	DATE
[1]	LAIRS Standard: Runtime-Governed Systems	White Paper v1.0 — Aegis Secure Convergence Systems	Feb 18, 2026
[2]	LAIRS Audit Fabric Standard	Technical Standard v2 — Russell Aydelotte	Feb 18, 2026
[3]	lairs-architecture-pub	ASConvergenceSystems / lairs-architecture-pub	Public
[4]	governed-adversarial-evolution	ASConvergenceSystems / governed-adversarial-evolution	Public

This document is an open, non-proprietary, non-confidential convergence of the LAIRS Runtime Governance Architecture and Audit Fabric Standard into a single unified public technical reference. Approved for public release. Licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International. Copyright © 2026 Russell Aydelotte, Aegis Secure Convergence Systems. <https://creativecommons.org/licenses/by-nc-sa/4.0>